

Autonomous Discovery Lab

Product Requirements Document — Claude Fable 5.1 Build Day, Mumbai, 23 Sep 2026

Team: Muaaz Shaikh (agent loop, sandbox, paper) + Teammate (live viewer, replay, paper renderer) · Track: Breakthrough · Budget: 90 min, USD 200 API credits

1. One-liner

Give the lab an open scientific question in plain English, e.g. *"find a cellular automaton rule under which a small seed self-replicates."* Fable 5.1 writes the experiments, runs thousands of simulations, grows a visible hypothesis tree with no human steering, then writes a short paper describing what it found, what failed, and why.

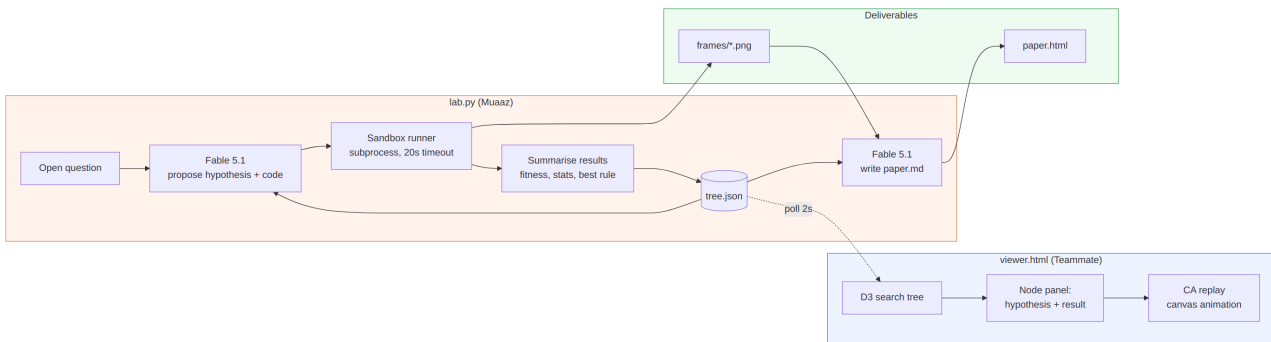
2. Why this is a Breakthrough-track entry

- **The claim:** a model can now run a coherent, multi-branch research program unsupervised. The tree is the evidence: each node must cite its parent, state a hypothesis, and be justified by the numbers that came before it.
- **Why cellular automata:** one experiment costs milliseconds, so in 30 minutes the bottleneck is the model's judgment, not compute. Thousands of runs are realistic on a laptop with no GPU.
- **What the last model choked on:** staying on-task past ~10 branches, not re-running the same rule family, and writing a paper that matches its own data. Fable 5.1 is tested on exactly that.
- **Novelty:** AI-scientist demos exist for ML papers with hours of GPU. A live, watchable search tree over a cheap substrate that finishes in a demo slot does not.

3. Goals and non-goals

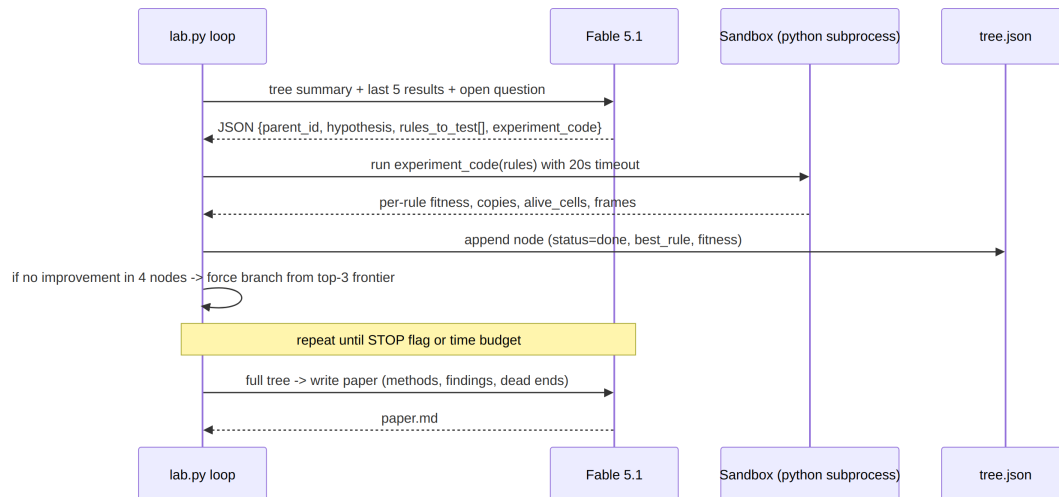
Goals (must ship by 20:10)	Non-goals (do not build tonight)
Loop runs unattended for 30+ min and produces 40+ tree nodes	Multiple substrates (only Life-like 2D CA tonight)
Live D3 tree that updates every 2 s and lets you click any node	Auth, multi-user, cloud deploy, Docker
Animated replay of the best rule found in any node	A real evaluator for 'true' self-replication — a proxy metric is fine
A paper.md written from the tree with embedded PNG frames	Fine-tuning, RAG, vector DBs, LangGraph — plain API calls only
A 60-second fallback screen recording	Perfect UI polish

4. System architecture



Two processes, one file as the contract. **lab.py** owns everything on the left and only ever appends to **tree.json**. **viewer.html** only ever reads it. Neither of you needs the other's code running to make progress.

5. The agent loop



- **Context stays small.** Fable never sees raw grids. It sees a compact summary of the tree (id, parent, hypothesis, best fitness) plus full results of the last 5 nodes. This keeps every call under ~15k tokens and the loop fast.
- **Experiment code is generated, not fixed.** Fable returns a Python function `run(rules, ca)` that uses the provided `ca` helpers. This is what makes it research rather than a parameter sweep: it can invent new seeds, new metrics, new rule families.
- **Anti-stagnation rule.** If the best fitness has not improved in 4 consecutive nodes, the loop forces Fable to branch from one of the top-3 frontier nodes instead of the latest one. Stops it from tunnelling.
- **Every node is durable.** Crash mid-run, restart, the loop resumes from `tree.json`. Do not lose 30 minutes of tree to a `KeyboardInterrupt`.

6. Data contract: tree.json

This is the only interface between the two of you. Agree on it in the first 10 minutes and do not change it after 19:00.

```

{
  "question": "Find a Life-like CA rule (B../S..) under which a 3x3 seed self-replicates.",
  "started_at": "2026-09-23T18:50:00+05:30",
  "status": "running", // running | writing_paper | done
  "best_node_id": "n17",
  "nodes": [
    {
      "id": "n0", "parent": null, "depth": 0,
      "hypothesis": "Baseline: sweep classic rules (Life, HighLife, Seeds) with a glider seed.",
      "rules_tested": ["B3/S23", "B36/S23", "B2/S"],
      "status": "done", // pending | running | done | error
      "started_at": "...", "ended_at": "...",
      "best_rule": "B36/S23",
      "fitness": 2.0, // copies of seed after N steps (proxy for replication)
      "metrics": {"copies": 2, "alive": 41, "growth": 1.3, "stable": true},
      "frames": ["frames/n0_0.png", "frames/n0_20.png", "frames/n0_40.png"],
      "replay": {"rule": "B36/S23", "seed": [[0,1,0],[0,1,1],[1,1,0]], "size": 48, "steps": 60},
      "notes": "HighLife produced a replicator-like growth; branch here.",
      "error": null
    }
  ]
}
  
```

7. Fitness: a proxy for self-replication

Do not try to detect 'true' replication tonight. Use a measurable proxy that rewards the right behaviour and let the agent refine it:

- Run the seed for N steps (default 60) on a 64x64 torus.
- **copies** = number of connected components whose normalised shape matches the seed (rotations/reflections allowed). This is the headline number.
- **growth** = alive cells at step N / alive cells at step 0. **stable** = population not exploding (< 30% of grid) and not dead.
- **fitness** = copies if stable else copies * 0.2. Explosive rules cannot win by accident.
- Fable is allowed to propose a better metric inside experiment_code, but must also report the standard one so nodes stay comparable.

8. Live viewer requirements

Element	Requirement
Tree	D3 tidy tree, root at left. Node colour = fitness (grey to orange). New nodes animate in. Poll tree.json every 2 s; never redraw from scratch, update in place.
Header	Question, node count, experiments run, elapsed time, best fitness so far. These four numbers are the pitch.
Node panel	Click a node: hypothesis, rules tested, metrics, notes, and the three PNG frames.
Replay	Canvas animation of node.replay (rule + seed), runs the CA in JS at 15 fps, loop. This is the money shot; make it big.
Status	Pulsing 'RUNNING' badge; switches to 'WRITING PAPER' then a 'Read paper' button linking to paper.html.
Constraints	Single HTML file, D3 from cdnjs, dark theme, works from file:// and from python -m http.server.

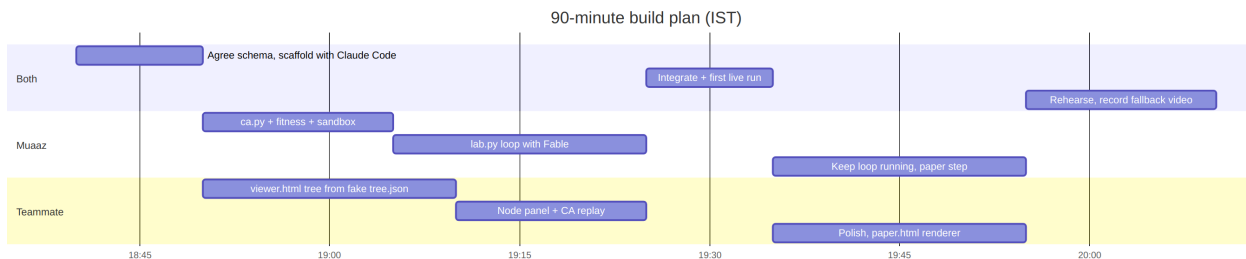
9. The paper

- Triggered by a STOP file, a time budget, or manually. Loop status becomes writing_paper.
- Input: the full tree (all nodes, notes, metrics) plus frame paths. Output: paper.md with sections Abstract, Question, Method, Search summary (with a table of the top 5 rules), Findings, Dead ends, Limitations, Next experiments.
- Hard requirement: every number in the paper must appear in tree.json. Fable is told this explicitly and asked to cite node ids inline, e.g. (n17).
- Teammate renders paper.md to paper.html with the frames inlined. Judges click one button and read it.

10. Demo script (2 minutes)

Time	What happens on screen	What you say
0:00	Live viewer, tree already 40+ nodes, RUNNING badge pulsing	"We gave it one sentence at 19:30. Nobody has touched it since. It has run N experiments."
0:25	Click root, then a dead-end branch	"Here it tried Seeds-type rules, saw explosions, wrote them off, and moved on. That's the coherence."
0:50	Click best node, replay plays big	"This is the rule it found. Watch the seed copy itself."
1:20	Open paper.html	"And it wrote this up, with its own dead ends, while we were talking."
1:45	Back to tree, a new node appears	"It's still going. The breakthrough is not the rule; it's hours of unsupervised, coherent research on a laptop."

11. Timeline



Hard rule: the loop must be running for real by 19:35 and never stopped after that. A 4-node tree loses; a 40-node tree wins.

12. Risks and mitigations

Risk	Mitigation
Fable returns broken experiment code	Sandbox catches exceptions, marks node error, feeds the traceback back as the next prompt's context. Errors are nodes too; they show coherence.
Loop tunnels on one rule family	Anti-stagnation branch rule (section 5). Also inject 'what family have you not tried?' every 8 nodes.
Nothing self-replicates	Proxy fitness rewards partial progress. The paper honestly reports 'closest candidates'. Judges reward the process.
Rate limits / API hiccup	Retry with backoff, max 3. Node stays pending; viewer shows it. Two API keys, one per teammate, as fallback.
Wi-Fi dies during demo	Viewer and tree are local files. Pre-recorded 60 s video as last resort.
Integration at 19:25 fails	Teammate builds against a hand-written fake tree.json from minute 10, so the viewer is demo-able even if the loop is late.

13. Credit budget (USD 200)

- Loop call: ~12k input + ~2k output tokens. Even at premium pricing, well under USD 1 per node. 60 nodes ≈ USD 30–60.
- Paper call: one large call, ~40k input. ≈ USD 2–5.
- You will not run out. Prefer quality (bigger max_tokens, extended thinking on) over saving credits.

14. Definition of done

- `python lab.py --question '...'` runs for 30 min unattended and writes a tree with ≥ 40 nodes, ≥ 1000 rule evaluations.
- `viewer.html` shows the tree live, a clicked node's details, and an animated replay.
- `paper.html` exists, cites node ids, and contains at least one embedded frame.
- A 60-second screen recording is on the phone.